

ZAIN JORDAN CUSTOMER 360 AI WORKSHOP

Main Printable Handbook

Three-Day Developer Workshop Reference

LangChain Tools | SQL Agents | RAG | Multi-Agent Systems | MCP | Capstone Projects |
Codex Demo

Purpose of this handbook

This is a print-ready student and trainer reference. It connects the workshop story, class-by-class delivery notes, prompts, code snippets, exercises, demo flows, capstone guidance, and evaluation rubrics into one structured document.

Source material: uploaded Class 1 to Class 9 workshop decks and Codex project notes.

Table of Contents

Use this as the main navigation map for the printed handout.

1. How to use this handbook
2. Three-day delivery map
3. Full learning journey
4. Class 1 - Database foundations
5. Class 2 - LangChain tools and agents
6. Class 3 - Natural language SQL agent
7. Class 4 - RAG assistant
8. Class 5 - Multi-agent customer care copilot
9. Class 6 - MCP enterprise integration
10. Class 7 - Capstone planning
11. Class 8 - Capstone build sprint
12. Class 9 - Final presentations and evaluation
13. Prompt and code repository
14. Codex dashboard demo playbook
15. Capstone and evaluation reference

1. How to Use This Handbook

This document is meant to be read before, during, and after the three-day session.

The workshop is not a collection of random AI modules. It is one connected implementation journey. Students begin with a realistic telecom Customer 360 database and gradually build toward enterprise AI workflows. The same database is reused across SQL, LangChain tools, SQL agents, RAG, multi-agent systems, MCP, and capstone projects.

How students should use it

- Before class: read the story and key vocabulary.
- During class: follow the demo flow and copy prompts/code snippets.
- During exercises: use the checklists and output templates.
- During capstone: use the blueprint, build tracks, and evaluation rubric.

How trainers should use it

- Use each class section as a delivery script.
- Use slide-by-slide notes to avoid losing the teaching story.
- Use code and prompt repository as a quick reference.
- Use transition notes to connect one class to the next.

Important: this handbook keeps the concepts aligned to the uploaded workshop slides. It expands the notes for readability, but does not introduce a different workshop architecture.

2. Three-Day Delivery Map

The nine classes can be delivered as a three-day learning journey.

The three-day structure groups the nine class decks into a practical delivery flow. Day 1 builds the data and agent foundations. Day 2 adds advanced AI architecture patterns. Day 3 moves teams into capstone planning, implementation, and final evaluation.

Day	Classes	Theme	Expected Outcome
Day 1	Classes 1-3	Data foundation, controlled tools, SQL agent	Students understand the Customer 360 database, build tool-using agents, and run natural-language analytics.
Day 2	Classes 4-6	RAG, multi-agent orchestration, MCP integration	Students connect SQL, retrieval, specialist agents, and reusable MCP tool access.
Day 3	Classes 7-9	Capstone planning, build sprint, final evaluation	Teams design, build, present, and evaluate working telecom AI prototypes.

Core delivery rhythm

- Explain the business problem first.
- Show the architecture pattern.
- Run a small demo.
- Make students inspect the output.
- Connect the pattern to capstone projects.

3. Full Learning Journey

The architecture evolves step by step from data to enterprise AI systems.

The full workshop journey starts with structured telecom data and ends with evaluated AI applications. Every class introduces one capability that becomes part of the final enterprise workflow.

Step	Workshop capability	Why it matters
1	Database exploration	Students learn what data exists before asking AI to use it.
2	Reusable Python functions	Database logic becomes testable and repeatable.
3	LangChain tools	Functions become controlled capabilities for agents.
4	SQL Agent	Natural language questions become structured analytics.
5	RAG Assistant	Semantic recommendations become grounded in retrieved evidence.
6	Multi-Agent Copilot	Specialist agents coordinate larger customer-care workflows.
7	MCP Integration	Approved tools become reusable enterprise capabilities.
8	Capstone Build	Teams apply the patterns to real telecom business problems.
9	Evaluation	Projects are reviewed for business value, database grounding, architecture clarity, and demo quality.

4. Class 1 - Getting Started with the Telecom Customer 360 Database

This class creates the foundation. Students first understand the telecom Customer 360 database before any agent or RAG layer is introduced. The key message is that useful AI starts with useful data understanding.

Learning outcomes

- Explain the Customer 360 story and major business domains.
- Connect to the SQLite database using Python.
- Inspect tables, row counts, sample records, and schemas.
- Run simple business SQL questions and first joins.
- Create reusable functions that later become LangChain tools and MCP tools.

Recommended delivery flow

- 1. Start with why telecom data matters for AI.
- 2. Orient students to the database: tables, rows, business areas.
- 3. Set up Google Colab, sqlite3, pandas, and the database path.
- 4. Run simple SQL queries: customers by city, plan catalog, churn risk, complaints, campaigns.
- 5. Run a first business join across customers, subscriptions, and plans.
- 6. Create reusable Customer 360 functions and connect them to future agent work.

Transition note

Class 1 ends by showing that these Python functions will become AI-callable tools in Class 2.

Class 1 reference notes

Business area	Representative tables	Training value
Customer profile	customers, accounts, subscriptions	Who the customer is and what they use
Billing and usage	invoices, payments, data/call/SMS usage	Revenue, behavior, and service usage
Customer experience	support, complaints, satisfaction	Pain points, sentiment, and service quality
Risk and value	churn scores, value segments	Retention and prioritization decisions
Network and campaigns	towers, events, campaigns	Proactive service and personalized offers

Code and prompt references from this class

Class 1 - Environment setup

```
import sqlite3
import pandas as pd
from pathlib import Path
import os

DB_PATH = "zain_customer_360_ai_demo.db"
conn = sqlite3.connect(DB_PATH)
print("Connected successfully")
```

Class 1 - Inspect database tables

```
query = """
SELECT name
FROM sqlite_master
WHERE type = 'table'
ORDER BY name;
"""

tables_df = pd.read_sql_query(query, conn)
tables_df
```

Class 1 - First business join

```
SELECT
  c.customer_id,
  c.full_name,
  c.city,
  s.msisdn,
  s.service_type,
  p.plan_name,
  p.monthly_fee_jod
FROM customers c
JOIN subscriptions s ON c.customer_id = s.customer_id
JOIN plans p ON s.plan_id = p.plan_id
LIMIT 10;
```

Class 1 - Reusable customer profile function

```
def get_customer_profile(customer_id):
    query = """
    SELECT c.customer_id, c.full_name,
           c.city, c.customer_segment,
           c.preferred_language, c.status
    FROM customers c
    WHERE c.customer_id = ?
    """
    return pd.read_sql_query(query, conn, params=(customer_id,))
```

Class 1 - First Customer 360 function

```
def get_customer_360(customer_id):
    return {
        "profile": get_customer_profile(customer_id),
        "plans": get_customer_plan(customer_id),
        "churn": get_customer_churn_risk(customer_id),
        "complaints": get_customer_complaints(customer_id),
        "support": get_customer_support_interactions(customer_id)
    }
```


Slide-by-slide delivery notes - Class 1

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Zain Jordan

Customer 360 AI Workshop | Class 1: Getting Started with the Telecom Customer 360 Database | Beginner-friendly foundation for Python, SQL, agents, RAG, MCP and capstone projects. | tables | 1K | customers | 8+ | use cases

Slide 2: Class 1 Outcomes

What participants should be able to do by the end of the session | Understand the dataset | Explain the telecom Customer 360 story and how tables connect to business questions. | Connect with Python | Use sqlite3 and pandas to open the database, inspect tables and preview records. | Ask SQL questions | Run simple queries for customers, plans, churn, complaints, network and campaigns. | Use joins | Combine customers, subscriptions and plans to answer realistic telecom questions.

Slide 3: Why This First Class Matters

Strong AI applications start with strong data understanding | Data | Customer, usage, billing, support and network records | Functions | Reusable Python queries that answer business questions | Tools | Functions the AI agent can call when needed | Agents | AI workflows that investigate and recommend action

Slide 4: The Zain Jordan Customer 360 Dataset

A synthetic telecom database designed for AI application development | database tables | 1,000 | customers | 1,631 | subscriptions | 60K | data sessions | 4,542

Slide 5: How to Explain Customer 360

A complete view of the customer across profile, behavior, service and value | Customer 360 | Profile | Customer identity, city, segment, language and account status | Products | Plans, subscriptions, SIMs, devices and activated add-ons | Experience | Support interactions, complaints, satisfaction and feedback | Business value

Slide 6: Class 1 Learning Flow

A practical sequence for beginners | Business context | Why telecom data matters for AI | Database orientation | Tables, row counts and categories | Python setup | sqlite3, pandas and database path | Simple SQL | SELECT, GROUP BY and ORDER BY

Slide 7: Environment Setup

Keep the technical setup simple and reliable | Recommended setup | Google Colab | SQLite database file | No local installation required | OpenAI/API keys only in later classes | import sqlite3 | import pandas as pd | from pathlib import Path

Slide-by-slide delivery notes - Class 1 (continued)

Slide 8: Hands-on 1: Inspect the Database

Participants learn what exists before asking business questions | query = `""" | SELECT name | FROM sqlite_master | WHERE type = 'table' | ORDER BY name; | """` | `tables_df = pd.read_sql_query(query, conn)` | What this teaches

Slide 9: Hands-on 2: Ask Simple Business Questions

Move from technical queries to business understanding | Customers by city | Which cities have the most customers? | Customers by segment | What types of customers does the company serve? | Plan catalog | Which plans have the highest monthly fees? | Churn risk | How many customers are high, medium or low risk?

Slide 10: Hands-on 3: First Business Join

Connect customer, subscription and plan data | `SELECT | c.customer_id, | c.full_name, | c.city, | s.msisdn, | s.service_type, | p.plan_name, | p.monthly_fee_jod`

Slide 11: Hands-on 4: High Value + High Churn Risk

The first "AI project idea" moment | Business question | Which high-value customers are also at high risk of churn? | Tables used | customers | customer_churn_scores | customer_value_segments | Business action | Prioritize retention outreach for customers with high value and high risk.

Slide 12: Create Reusable Database Functions

Prepare Python functions that later become tools | `def get_customer_profile(customer_id): | query = """ | SELECT c.customer_id, c.full_name, | c.city, c.customer_segment, | c.preferred_language, c.status | FROM customers c | WHERE c.customer_id = ? | """`

Slide 13: Build the First Customer 360 Function

Combine profile, plan, churn, complaints and support history | `def get_customer_360(customer_id): | return { | "profile": get_customer_profile(customer_id), | "plans": get_customer_plan(customer_id), | "churn": get_customer_churn_risk(customer_id), | "complaints": get_customer_complaints(customer_id), | "support": get_customer_support_interactions(customer_id) | }`

Slide 14: Exercises and Wrap-up

Convert learning into hands-on practice | Exercise | Expected Output | Top 10 cities by customer count | City-level customer distribution | Top 10 customers by churn score | Prioritized churn risk list | Complaint categories by frequency | Service pain-point summary

Slide-by-slide delivery notes - Class 1 (continued)

Slide 15: Class 1 Closing Script

Suggested ending for the trainer | Today, we completed the foundation. | Summary | We connected Python to the Zain Jordan Customer 360 database, explored telecom data, asked business questions using SQL, and created reusable Python functions. | Transition | These functions are not just Python practice. They are the building blocks for LangChain tools, AI agents and MCP-enabled enterprise workflows.

5. Class 2 - Building AI Agents with LangChain Tools

This class turns database functions into controlled AI tools. Students move from normal Python functions into a simple Customer Care AI Agent that can decide which tool to call.

Learning outcomes

- Explain the difference between chatbot and agent.
- Convert Python functions into LangChain tools.
- Understand why tool descriptions matter.
- Build a Customer Care AI Agent.
- Produce structured customer-care recommendations from database-backed evidence.

Recommended delivery flow

- 1. Recap Class 1 functions.
- 2. Explain chatbot versus agent.
- 3. Show the controlled architecture: user question -> agent -> tools -> database.
- 4. Convert functions to tools.
- 5. Build the first two-tool agent.
- 6. Expand into the full customer-care agent.
- 7. Practice tool selection and customer analysis.

Transition note

Class 2 gives controlled customer workflows. Class 3 expands to flexible business analytics through SQL agents.

Class 2 tool reference

Tool	Purpose
get_customer_profile	Customer identity, city, segment, language, account status
get_customer_plan	Subscriptions, plan name, monthly fee, allowances
get_customer_churn_risk	Churn score, risk level, reason, recommended action
get_customer_complaints	Recent complaint categories, severity, status, compensation
get_customer_support_interactions	Support history, channel, issue, priority, sentiment
get_customer_billing_summary	Invoices, payment status, overdue days
get_customer_usage_summary	Data sessions, total GB, cost, last activity

Code and prompt references from this class

Class 2 - Convert function into LangChain tool

```
@tool
def get_customer_churn_risk(customer_id: int) -> str:
    """Get churn score, risk level, reason, and recommended action."""
    # Call database function and return readable output
    return fetch_customer_churn_risk(customer_id)
```

Class 2 - Main Customer Care Agent prompt

Analyze customer 42. Check profile, plan, churn risk, recent complaints, support history, billing, and usage. Recommend the next best action and draft a short message.

Class 2 - First two-tool demo prompt

Who is customer 42 and what is their churn risk?

Slide-by-slide delivery notes - Class 2

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Building AI Agents

with LangChain Tools | Using the Zain Jordan Customer 360 Database | Python Functions → LangChain Tools → AI Agent | Customer Care Use Case | Customer Care AI Agent | Profile | Plan | Risk | Billing

Slide 2: Where This Class Fits

A smooth path from database foundations to enterprise AI integration | Database + Python Basics | LangChain Tools + Agent | SQL Agent | RAG | Class 5+ | Multi-Agent + MCP | Class 2 Focus | Convert the database functions built in Class 1 into tools an AI agent can call. The agent will use controlled access to customer data and produce a structured customer-care recommendation.

Slide 3: Learning Objectives

By the end of Class 2, participants should be able to explain and build a simple tool-using agent | Concepts | - Difference between chatbot and agent | - What a tool is in LangChain | - Why tool descriptions matter | - Why controlled tools are safer than open database access | Hands-On Skills | - Convert Python functions into LangChain tools | - Create a tool-using AI agent

Slide 4: Chatbot vs Agent

The key conceptual shift for participants | Normal Chatbot | Responds directly | Often text-only | May guess when data is missing | Does not decide what action to take | Limited workflow control | AI Agent | Decides what information is needed

Slide 5: Class 2 Architecture

A controlled, beginner-friendly agent design | User Question | AI Agent | LangChain Tools | Python DB Functions | SQLite Database | → | Why This Design Works | The agent does not get unlimited database access. It can only call the tools we provide. This keeps the demo understandable, controlled, and suitable for enterprise training.

Slide 6: Tools Used in Class 2

Each tool wraps a database function from Class 1 | Tool | Purpose | get_customer_profile | Customer identity, city, segment, language, account status | get_customer_plan | Subscriptions, plan name, monthly fee, allowances | get_customer_churn_risk | Churn score, risk level, main reason, recommended action

Slide 7: Main Use Case: Customer Care AI Agent

One agent, multiple customer-care tools | Business Prompt | Analyze customer 42. Check profile, plan, churn risk, recent complaints, support history, billing, and usage. Recommend the next best action and draft a short message. | Agent Task | Call the right tools, combine the results, avoid guessing, and produce a structured customer-care recommendation. | Profile | Plan | Risk | Complaints

Slide-by-slide delivery notes - Class 2 (continued)

Slide 8: Notebook Setup

What participants configure before running the agent | Required Packages | langchain | langchain-openai | langchain-community | pandas | Required Secrets | OPENAI_API_KEY | Stored in Colab Secrets

Slide 9: Step 1: Reuse Database Functions

The agent starts with normal Python functions | Class 1 Function Pattern | Input: customer_id | Action: run SQL query | Output: readable text or table result | Example Function | fetch_customer_profile(customer_id) | fetch_customer_plan(customer_id) | fetch_customer_churn_risk(customer_id)

Slide 10: Step 2: Convert Functions into LangChain Tools

Tool names and descriptions guide the agent | Tool Name | A short, specific name such as get_customer_churn_risk. The model uses this to identify the tool. | Tool Description | A precise description of when the tool should be used and what it returns. | Tool Output | Clear text or structured data. For beginners, readable text is easier than complex objects. | @tool | def get_customer_churn_risk(customer_id: int) -> str:

Slide 11: Step 3: Build the First Agent

Start with only two tools | get_customer_profile | get_customer_churn_risk | Basic Agent | Final Answer | → | First Demo Prompt | "Who is customer 42 and what is their churn risk?" | Expected Behavior

Slide 12: Step 4: Full Customer Care Agent

Expand the agent only after the first version works | Tools Available | - Profile | - Plan | - Churn risk | - Complaints | - Support interactions | - Billing summary | - Usage summary

Slide 13: Structured Output Format

A predictable answer format makes the agent useful for business teams | Section | What It Should Contain | Customer Summary | Name, city, segment, preferred language, account status | Plan Summary | Current plan, fee, data allowance, service type | Risk Summary | Churn score, risk level, main risk reason

Slide 14: Participant Exercises

Hands-on practice after the main demo | Exercise 1 | Tool Selection | Match user questions to the correct tool: profile, plan, churn, complaints, support, billing, or usage. | Exercise 2 | Customer Analysis | Analyze customer 10, 25, 42, and 100. Identify who needs urgent follow-up. | Exercise 3 | Prompt Improvement

Slide-by-slide delivery notes - Class 2 (continued)

Slide 15: Common Errors and Quick Fixes

Practical troubleshooting for the live class | Issue | Likely Fix | OPENAI_API_KEY not found | Add key in Colab Secrets and enable notebook access | Database file not found | Upload .db file and confirm DB_PATH exists | Tool returns no records | Try another customer ID or explain that data is unavailable

Slide 16: Class 2 Wrap-Up

The foundation for SQL agents, RAG, multi-agent workflows, and MCP | Python Functions | LangChain Tools | Single Agent | Customer-Care Recommendation | → | What Participants Built | A tool-using customer-care agent that can retrieve customer data, assess risk signals, and generate a business-friendly recommendation. | Next Class

6. Class 3 - Natural Language SQL Agent for Telecom Business Questions

This class shifts from predefined tools to flexible analytics. Students learn how a SQL agent can inspect a database, generate SQL, execute the query, and explain the result in business language.

Learning outcomes

- Compare tools agents and SQL agents.
- Connect LangChain to SQLite through SQLDatabase.
- Create SQLDatabaseToolkit components.
- Ask telecom business questions in natural language.
- Use guardrails for SELECT-only, limited, schema-aware querying.

Recommended delivery flow

- 1. Explain why managers ask business questions in English but data lives in SQL.
- 2. Compare tools agents with SQL agents.
- 3. Show the SQL agent architecture.
- 4. Warm up with manual SQL.
- 5. Build SQLDatabase, toolkit, and agent.
- 6. Run simple, intermediate, and business-ready questions.
- 7. Discuss safety and production guardrails.

Transition note

Class 3 handles exact calculations and structured analytics. Class 4 adds RAG for semantic recommendations and contextual answers.

Class 3 demo question ladder

Question level	Example questions
Beginner	How many customers? Customers by city. Active vs inactive. Plans by monthly fee. Churn risk counts.
Intermediate	High churn by city. Average churn by segment. ARPU by value segment. Top complaint categories.
Business-ready	High value + high churn. Campaign conversion rate. Network events by impact. Complaint + churn overlap. Executive summary.

Code and prompt references from this class

Class 3 - Manual SQL warm-up

```
SELECT c.city, COUNT(*) AS high_risk_customers
FROM customers c
JOIN customer_churn_scores ch
  ON c.customer_id = ch.customer_id
WHERE ch.risk_level = 'High'
GROUP BY c.city
ORDER BY high_risk_customers DESC
LIMIT 10;
```

Class 3 - SQL Agent rules prompt

You are a telecom BI SQL agent.

Rules:

- Use SELECT queries only.
- Never modify the database.
- Inspect schemas before querying.
- Limit results unless requested.
- Do not guess missing facts.
- Explain results in business language.

Final answer:

1. Direct answer
2. Key numbers
3. Business interpretation
4. Recommended next action

Slide-by-slide delivery notes - Class 3

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Natural Language SQL Agent

for Telecom Business Questions | Using the Zain Jordan Customer 360 SQLite Database | Today we move from | controlled tools to | flexible database analytics. | SQL Agent = natural language → query → insight

Slide 2: Where Class 3 Fits

From database access to AI-powered business intelligence. | Database | Tools Agent | SQL Agent | RAG | Multi-Agent | MCP | What changes today? | Class 2 agents used controlled Python tools. Class 3 agents can inspect the database and generate SQL for wider business questions.

Slide 3: Class 3 Learning Outcomes

By the end of this session, participants should be comfortable with the SQL agent workflow. | Understand the difference between a tools agent and a SQL agent. | Connect LangChain to the SQLite Customer 360 database. | Create SQLDatabase and SQLDatabaseToolkit components. | Ask telecom business questions in natural language. | Explain SQL agent guardrails and production safety risks. | Generate capstone-ready analytics use cases.

Slide 4: Class 2 Recap

The previous class gave the agent controlled tools. | What we built | - Python DB functions | - LangChain tool wrappers | - Customer Care AI Agent | - Retention-focused agent | Best for | Customer-specific workflows: | - Analyze customer 42

Slide 5: Why a SQL Agent?

Business users ask questions in English, but enterprise data lives in relational databases. | The business problem | Managers often need answers from the database, but they may not know table names, joins, or SQL syntax. | Examples: | - Which cities have high churn? | - Which campaigns converted best? | - Which complaint categories are growing? | The SQL agent solution | The agent translates a natural-language question into a database query, runs the query, and explains the result in business language.

Slide 6: Tools Agent vs SQL Agent

Both are useful, but they solve different kinds of problems. | Class 2: Tools Agent | Access style | Uses predefined Python functions. | Best for | Controlled customer workflows. | Example | "Analyze customer 42." | Strength

Slide 7: SQL Agent Architecture

The agent follows a controlled sequence to answer a question. | User | Question | List | Tables | Inspect | Schema | Generate | SQL

Slide-by-slide delivery notes - Class 3 (continued)

Slide 8: Zain Jordan Database Areas

The SQL agent can explore multiple telecom business domains. | Customer Core | customers | accounts | subscriptions | Products | plans | addons | subscription_addons

Slide 9: Manual SQL Warm-Up

Start with a normal query before asking the agent to generate one. | BUSINESS QUESTION | Which cities have the most high-risk churn customers? | `SELECT c.city, COUNT(*) AS high_risk_customers | FROM customers c | JOIN customer_churn_scores ch | ON c.customer_id = ch.customer_id | WHERE ch.risk_level = 'High' | GROUP BY c.city`

Slide 10: Building Blocks of the SQL Agent

Five components create the end-to-end natural language SQL experience. | SQLite Database | The local Zain Jordan Customer 360 `.db` file. | SQLDatabase | LangChain wrapper that exposes schema and query access. | Chat Model | The LLM that decides next steps and explains results. | SQL Toolkit | Tools for list tables, inspect schema, check query, and execute query.

Slide 11: Notebook Flow

How the Class 3 notebook is organized. | Setup | Install libraries, load key, upload DB | Warm-Up | Manual SQL queries for business context | SQLDatabase | Connect LangChain to SQLite | Toolkit | Create database tools for the agent

Slide 12: Core Demo Questions

Start simple, then move into real telecom analytics. | Beginner | - How many customers? | - Customers by city | - Active vs inactive | - Plans by monthly fee | - Churn risk counts | Intermediate | - High churn by city

Slide 13: Demo Path: From Question to Insight

Use one question to explain the full cycle. | Question | "Which customer value segments have the highest average ARPU and six-month revenue?" | Inspect | tables | Choose | value table | Generate | GROUP BY SQL

Slide 14: SQL Agent Safety

The most important professional habit in this class. | Training environment | - Synthetic data | - Local SQLite file | - Read-only style prompt | - Row limits | - No destructive actions | - Visible agent steps during demo | Production guardrails

Slide-by-slide delivery notes - Class 3 (continued)

Slide 15: Prompt Rules for the SQL Agent

The system prompt controls safety, accuracy, and answer quality. | You are a telecom BI SQL agent. | Rules: | - Use SELECT queries only. | - Never modify the database. | - Inspect schemas before querying. | - Limit results unless requested. | - Do not guess missing facts. | - Explain results in business language.

Slide 16: Participant Exercises

Move learners from running demos to asking useful questions. | Exercise 1 | Simple questions | - How many customers? | - How many plans? | - How many complaints? | - How many network events? | Exercise 2 | Grouped analysis

Slide 17: Common Mistakes to Teach

Make participants better prompt writers and better reviewers. | Vague question | "Tell me about customers." | Too much at once | "Analyze everything." | No row limit | "Show all customers." | Blind trust | Not checking tables, filters, or logic.

Slide 18: Capstone Project Patterns

Day 3 gives teams flexible analytics capability for their final builds. | Executive KPI Assistant | Revenue, churn, complaints, campaigns, network overview | Churn Analytics Assistant | Risk segments, high-value churn, city-level patterns | Complaint Intelligence Assistant | Top categories, severity, sentiment, repeat issues | Campaign Performance Assistant | Conversion rate, target segment, offer effectiveness

Slide 19: What Class 3 Produces

A working analytics assistant plus capstone-ready thinking. | SQLDatabase | SQL Toolkit | SQL Agent | BI Demos | Safety | Capstone Ideas | End-of-class capability | Participants can ask structured business questions about churn, revenue, complaints, campaigns, support interactions, and network events using natural language. They also understand why guardrails are mandatory before production use.

Slide 20: Closing and Next Step

Class 3 completes the analytics foundation. | Today's milestone | We moved from controlled customer tools to flexible database analytics. The SQL agent can inspect the database, generate SQL, execute queries, and explain business results. | Next class | RAG from telecom database content. We will convert selected rows into searchable knowledge for plan, campaign, support, and customer-care recommendations. | Core message: SQL Agent gives flexible analytics. RAG gives contextual knowledge retrieval.

7. Class 4 - Building a RAG Assistant from Telecom Database Content

This class teaches when SQL is not enough. Students learn to convert meaningful database rows into searchable documents so the LLM can produce grounded recommendations.

Learning outcomes

- Explain indexing, retrieval, and generation.
- Choose which database tables are useful for RAG.
- Convert rows into text documents with metadata.
- Create embeddings and an in-memory vector store.
- Test retrieval before generation.
- Build a grounded plan, offer, or support recommendation assistant.

Recommended delivery flow

- 1. Compare SQL agent and RAG assistant.
- 2. Explain the RAG flow.
- 3. Select content-rich tables.
- 4. Build documents from rows.
- 5. Create embeddings and vector store.
- 6. Test retrieval quality.
- 7. Generate grounded recommendations and connect SQL + RAG as a hybrid pattern.

Transition note

Class 4 builds the semantic recommendation layer. Class 5 combines SQL, tools, RAG, and messaging through multi-agent orchestration.

Class 4 RAG source selection

Table	Text to retrieve	Use case
plans	Plan names, fees, data, minutes, contract terms	Plan recommendation assistant
addons	Extra data, roaming, voice, SMS bundles	Add-on recommendation
campaigns	Offer descriptions and target segments	Campaign matching
complaints	Complaint category, descriptions, resolution summaries	Complaint theme search
support_interactions	Issues, channels, sentiment, resolution notes	Support guidance
customer_satisfaction	NPS, CSAT, feedback text, sentiment	Customer experience insights

Code and prompt references from this class

Class 4 - Build documents from database rows

```
for row in plans:
    text = f"""
    Plan: {row.plan_name}
    Category: {row.plan_category}
    Monthly Fee: {row.monthly_fee_jod} JOD
    Data: {row.data_allowance_gb} GB
    Minutes: {row.local_minutes}
    """
    documents.append(Document(
        page_content=text,
        metadata={"table": "plans", "id": row.plan_id}
    ))
```

Class 4 - Create embeddings and vector store

```
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = InMemoryVectorStore(embeddings)
vector_store.add_documents(documents)
retriever = vector_store.as_retriever(search_kwargs={"k": 4})
```

Class 4 - Retrieval before generation

```
query = "best offer for a heavy data user with roaming needs"
results = retriever.invoke(query)

for doc in results:
    print(doc.metadata)
    print(doc.page_content[:500])
```

Class 4 - RAG answer format

```
Final answer format:
1. Recommended plan / add-on / campaign
2. Why it fits the customer need
3. Evidence from retrieved documents
4. Confidence / limitations
5. Customer-care message
```

Slide-by-slide delivery notes - Class 4

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: RAG Assistant

from Telecom | Database Content | SQLite Database | Plans, campaigns, complaints, support notes, customer feedback | Documents | Rows become searchable text with metadata | Vector Store | Embeddings support semantic search | RAG Answer

Slide 2: Where Workshop 4 Fits

The learning path now moves from database querying to retrieval-based AI answers. | Database Foundation | Explore tables, columns, and business meaning | LangChain Tools Agent | Controlled tools for customer-specific workflows | SQL Agent | Natural language questions become SQL queries | RAG Assistant | Retrieve relevant content before generating answers

Slide 3: From SQL Agent to RAG Assistant

SQL Agent | Best for structured questions: | - counts and totals | - grouping and ranking | - joins across tables | - numeric business analysis | RAG Assistant | Best for meaning-based questions: | - plan suitability

Slide 4: What RAG Means

1. Indexing | Create documents from source content, split if needed, embed, and store vectors. | 2. Retrieval | For a user question, search semantically similar documents from the vector store. | 3. Generation | Pass retrieved context to the model and generate a grounded business answer. | In this class, source content comes from selected rows in the same Zain Jordan SQLite database.

Slide 5: Which Database Content Becomes RAG?

Not every table should become a vector document. Choose content that benefits from semantic search. | Table | Text to retrieve | Training use case | plans | Plan names, fees, data, minutes, contract terms | Plan recommendation assistant | addons | Extra data, roaming, voice, SMS bundles

Slide 6: Workshop 4 Architecture

SQLite DB | selected tables | Documents | row-to-text builder | Embeddings | semantic vectors | Vector Store | similarity search | RAG Answer

Slide 7: Row-to-Document Design

A database row must be converted into a clear text document before embedding. | Document text example | Plan Name: Family 50 | Category: Consumer Mobile | Monthly Fee: 50 JOD | Data Allowance: 120 GB | Local Minutes: Unlimited | Technology: 5G | Contract: 12 months

Slide-by-slide delivery notes - Class 4 (continued)

Slide 8: Class Build Plan

Select tables | Choose content-rich tables: plans, addons, campaigns, complaints, support notes | Create documents | Convert selected rows into readable text + metadata | Embed documents | Use embedding model to convert text into vectors | Store vectors | Use an in-memory vector store for beginner simplicity | Retrieve context

Slide 9: Demo 1 - Build Documents from Database Rows

The first demo converts structured telecom rows into text documents that retrieval can understand. | Core pattern | for row in plans: | text = f"" | Plan: {row.plan_name} | Category: {row.plan_category} | Monthly Fee: {row.monthly_fee_jod} JOD | Data: {row.data_allowance_gb} GB | Minutes: {row.local_minutes}

Slide 10: Demo 2 - Create Embeddings and Vector Store

Embeddings make semantic search possible: the assistant can find related content even when words do not match exactly. | Embedding model | Transforms text into numeric vectors that capture meaning. | Vector store | Stores vectors and returns semantically similar documents. | Retriever | Search interface that returns the top relevant documents. | Notebook concept | embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

Slide 11: Demo 3 - Retrieval Before Generation

Before asking the LLM to answer, we first inspect what documents retrieval returns. | Retrieval test | query = "best offer for a heavy data user with roaming needs" | results = retriever.invoke(query) | for doc in results: | print(doc.metadata) | print(doc.page_content[:500]) | Why test retrieval? | If retrieved documents are poor, the final answer will be poor. RAG quality begins with retrieval quality.

Slide 12: Demo 4 - Plan & Offer RAG Assistant

User request | Customer uses a lot of data and travels frequently. Recommend a suitable plan, add-on, or campaign. | Retriever context | Relevant plans, roaming add-ons, and campaign descriptions retrieved from the vector store. | RAG answer | Recommendation, reasoning, source references, and short customer-care message. | Output should always say what evidence it used. This is the difference between a useful assistant and a fluent guesser. | Structured response | Final answer format:

Slide 13: Hybrid Pattern - SQL + RAG

The best capstone projects can combine exact customer facts from SQL with semantic guidance from RAG. | SQL layer | Get customer facts: | - profile | - plan | - usage | - churn risk | - billing status | RAG layer

Slide 14: Grounded Answer Checklist

Relevant retrieval | Top documents match the user question | Source visibility | Answer can point back to table and record | No unsupported claims | No recommendation without evidence | Clear limitations | If data is missing, say so | Business action

Slide-by-slide delivery notes - Class 4 (continued)

Slide 15: Participant Exercises

Exercise 1: Retrieval test | Query the vector store for: | "best add-on for roaming customer" | Review the top 4 documents. | Exercise 2: Plan advisor | Ask the RAG assistant to recommend a plan for a heavy data user. | Exercise 3: Complaint theme | Retrieve support/complaint records for "billing confusion" and summarize patterns. | Exercise 4: Improve documents

Slide 16: Capstone Ideas Using RAG

Plan Recommendation Assistant | Recommends plans/add-ons using customer need and plan knowledge | Campaign Match Assistant | Matches customer segment to relevant campaigns and offers | Support Resolution Assistant | Finds similar complaint/support cases and suggests resolution path | Customer Message Assistant | Generates grounded care messages using retrieved policies/offers | Knowledge Search Assistant

Slide 17: Common RAG Mistakes

Vectorizing everything | The full database is not automatically useful. Select tables that contain meaning-rich text. | Weak document text | If rows are converted poorly, retrieval quality will suffer even with a good model. | No source tracking | Without metadata, users cannot verify why the assistant gave an answer. | Answering beyond context | The model should not invent policies, prices, or offers not present in retrieved context. | Skipping evaluation

Slide 18: What Learners Build Today

Documents | Plans, add-ons, campaigns, complaints, and support rows converted to text | Vector Store | Embeddings stored for semantic search | Retriever | Top relevant documents returned for user questions | RAG Assistant | Grounded answers for telecom plan, campaign, and support use cases | Capstone Foundation

Slide 19: Transition to the Next Workshops

Next: Multi-Agent | Specialist agents can use SQL and RAG as tools. Example: Customer Agent + Offer Agent + Support Agent. | Then: MCP | The same database and RAG functions can be exposed as standardized enterprise tools. | Final: Capstone | Teams build a practical telecom AI assistant using the shared Zain Jordan database. | The pattern is now complete: structured SQL insights + semantic RAG recommendations + agentic orchestration.

Slide 20: Reference Concepts

Workshop 4 uses standard retrieval-augmented generation concepts from the LangChain ecosystem. | Core RAG flow | Index source data, retrieve relevant context at question time, then generate an answer from the retrieved context. | Retrieval building blocks | Document loaders, text splitters, embeddings, vector stores, and retrievers. | Workshop adaptation | Instead of external documents, we create documents from selected telecom database rows. | Source note: Official LangChain RAG, retrieval, embeddings, and vector store documentation were used to align the workshop language.

8. Class 5 - Multi-Agent Customer Care Copilot

This is the integration class. Students learn why one agent with many responsibilities can become hard to manage and how specialist agents can be coordinated by a supervisor.

Learning outcomes

- Understand why complex workflows benefit from specialist agents.
- Define Customer Data, SQL Analyst, RAG Recommendation, and Care Message agents.
- Use a Supervisor Agent for routing and synthesis.
- Produce one final customer-care report.
- Connect prior classes into a complete enterprise workflow.

Recommended delivery flow

- 1. Review tools, SQL agent, and RAG.
- 2. Explain why one agent becomes overloaded.
- 3. Introduce specialist agent responsibilities.
- 4. Explain supervisor routing rules.
- 5. Build and test each specialist separately.
- 6. Run full customer-retention and care demos.
- 7. Map multi-agent patterns to capstone projects.

Transition note

Class 5 coordinates components inside the notebook. Class 6 exposes selected capabilities through MCP for enterprise-style reuse.

Class 5 specialist-agent map

Agent	Responsibility
Customer Data Agent	Customer-specific facts: profile, plan, churn, value, complaints, support, billing, usage.
SQL Analyst Agent	Broad business context: churn by city, complaint patterns, revenue, campaign insights.
RAG Recommendation Agent	Semantic recommendations from plans, add-ons, campaigns, and support themes.
Care Message Agent	Customer-facing message and internal care note.
Supervisor Agent	Coordinates specialists and produces final structured customer-care report.

Code and prompt references from this class

Class 5 - Multi-agent demo prompt

Analyze customer 42. Check profile, plan, churn risk, value, billing, complaints, support history, usage, relevant offers, then draft a professional care message.

Slide-by-slide delivery notes - Class 5

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Multi-Agent Customer

Care Copilot | Combining customer data, SQL analysis, RAG recommendations, and care messaging into one coordinated AI workflow. | Zain Jordan Customer 360 Database | LangChain Multi-Agent Pattern | Workshop Integration Class | Supervisor | Agent | Customer Data | SQL Analyst

Slide 2: Where Class 5 Fits

CONTEXT | This is the integration class where prior components become one customer-care workflow. | Database + Python | SQLite exploration | LangChain tools | Single agent | Natural language | SQL agent | RAG assistant

Slide 3: Class 5 Goal

GOAL | Build a practical multi-agent customer-care system around the Zain Jordan database. | What participants will understand | - Why one large agent can become difficult to manage | - How to split responsibilities across specialist agents | - How a supervisor coordinates subagents | - How prior classes connect into a real system | What participants will build | - Customer Data Agent

Slide 4: Why Move Beyond One Agent?

WHY | As workflows grow, one agent with many tools becomes harder to reason about and debug. | One-agent pressure | The same agent must retrieve customer details, perform SQL analysis, search recommendations, reason about churn, and write customer messages. This works for simple demos but becomes hard to inspect as complexity grows. | Multi-agent advantage | Each specialist agent owns a clear responsibility. The supervisor asks the right agent at the right time, then combines the evidence into a complete customer-care report. | Design principle: split by responsibility, not by hype.

Slide 5: One Agent vs Multi-Agent

COMPARISON | Use one agent for simple workflows; use multi-agent when responsibilities become distinct. | One Agent | - Many tools in one place | - Fast to prototype | - Good for simple tasks | - Can become overloaded | - Harder to debug decisions | - Less modular for capstones

Slide 6: Multi-Agent Customer Care Architecture

ARCHITECTURE | A supervisor coordinates four specialist agents and produces the final report. | User Request | Analyze customer 42 | Supervisor Agent | Routes + synthesizes | Customer Data | Agent | SQL Analyst

Slide 7: Specialist Agent Responsibilities

ROLES | Each agent has a clearly defined job and output. | Customer Data Agent | Customer-specific facts: profile, plan, churn, value, complaints, support, billing, usage. | SQL Analyst Agent | Broad business context: churn by city, complaint patterns, revenue and campaign insights. | RAG Recommendation Agent | Semantic recommendations: plans, add-ons, campaigns, complaint themes, support feedback. | Care Message Agent

Slide-by-slide delivery notes - Class 5 (continued)

Slide 8: Main Class Demo: Customer Retention and Care Copilot

USE CASE | The system analyzes a customer and turns evidence into a recommended action. | Demo prompt | Analyze customer 42. Check profile, plan, churn risk, value, billing, complaints, support history, usage, relevant offers, then draft a professional care message. | Inputs | Customer ID | Business question | Requested output sections | Agent work

Slide 9: Customer Data Agent

AGENT 1 | Retrieves customer-specific evidence from the Zain Jordan Customer 360 database. | Purpose | Get a complete customer 360 summary for a specific customer ID. It should not perform broad analytics or invent data. | Tools used | - get_customer_profile | - get_customer_plan | - get_customer_churn_risk | - get_customer_complaints

Slide 10: SQL Analyst Agent

AGENT 2 | Answers broader business questions using the natural language SQL agent pattern. | Best for | - Churn by city | - ARPU by segment | - Complaint volume | - Campaign conversion | - Network event impact | - Executive-style insights

Slide 11: RAG Recommendation Agent

AGENT 3 | Finds relevant context from plans, add-ons, campaigns, and customer-experience records. | RAG-friendly data | - plans | - addons | - campaigns | - complaints | - support_interactions | - customer_satisfaction

Slide 12: Care Message Agent

AGENT 4 | Transforms analysis into clear, professional communication. | Customer-facing message | Short, polite, professional, under 120 words, no internal churn-score language, no overpromised compensation. | Internal note | Brief follow-up note for the care team: issue, action, escalation need, and what should be checked next. | Why separate? | Communication quality matters. A dedicated message agent helps maintain tone, clarity, and policy-safe language.

Slide 13: Supervisor Agent

SUPERVISOR | Coordinates specialist agents and produces the final customer-care report. | Responsibilities | - Understand the request | - Decide which agent to call | - Collect outputs | - Resolve evidence into a report | - Keep answer structured | Routing rules

Slide 14: How the Full Workflow Runs

FLOW | The class notebook builds each component, tests it, then integrates under a supervisor. | Load DB + model | Build customer tools | Create SQL analyst | Build RAG knowledge base | Wrap agents as tools | Run supervisor demo | Training principle

Slide-by-slide delivery notes - Class 5 (continued)

Slide 15: Notebook Structure

NOTEBOOK | The class notebook mirrors the architecture in a beginner-friendly sequence. | Part A | Setup | API key, database, model, helper functions | Part B | Specialists | Customer, SQL, RAG, Care Message agents | Part C

Slide 16: Class Demo Flow

DEMO | Four demos move from one customer analysis to comparison and business context. | Demo 1 | Analyze customer 42 with all specialist agents and produce a full customer-care report. | Demo 2 | Retention-focused analysis for customer 25: risk, complaints, offer, and care message. | Demo 3 | Compare customers 25 and 42 to prioritize retention team action. | Demo 4

Slide 17: Expected Final Customer-Care Report

OUTPUT | The supervisor composes a structured answer that is useful for customer-care teams. | 1. Customer Summary | 2. Key Issue Diagnosis | 3. Churn and Value Risk | 4. Evidence from Data | 5. Recommended Next Action | 6. Suggested Offer / Plan / Campaign | 7. Customer-Care Message

Slide 18: Participant Exercises

PRACTICE | Exercises reinforce role selection, customer analysis, and capstone mapping. | Exercise 1: Right agent | Match user request to the best agent: customer facts, business trend, recommendation, or message writing. | Exercise 2: Customer analysis | Run the supervisor for customers 10, 25, 42, and 100. Compare urgency, risk, complaints, billing, and recommended actions. | Exercise 3: Capstone mapping | Choose a capstone idea and define required agents, tables, tools, final output, and demo script.

Slide 19: Capstone Connection

CAPSTONE | Class 5 provides a reusable architecture for team projects. | Churn Rescue Copilot | Prioritize high-risk, high-value customers and recommend retention action. | Billing Support Agent | Explain invoices, payment status, and next support action. | Campaign Recommendation Agent | Match segments to offers and draft outreach messages. | Network Impact Agent

Slide 20: RISKS

Common Mistakes in Multi-Agent Systems | A strong architecture depends on clear roles and disciplined testing. | Design mistakes | - Too many agents too early | - Unclear agent responsibilities | - Supervisor doing all work | - Poor tool descriptions | Testing mistakes | - Subagents not tested separately

Slide 21: NEXT

What Comes Next: MCP | After building multi-agent workflows, the next step is enterprise tool integration. | Python | Functions | LangChain | Tools | Agent | Workflow | Multi-Agent

Slide-by-slide delivery notes - Class 5 (continued)

Slide 22: Class 5 Summary

A multi-agent system works best when each agent has a clear responsibility and the supervisor coordinates the final workflow. |
Built today | Customer Data Agent | SQL Analyst Agent | RAG Recommendation Agent | Care Message Agent | Supervisor Agent |
Business value | Faster diagnosis

9. Class 6 - MCP for Enterprise AI Tool Integration

This class moves from local notebook tools into reusable enterprise-style tool access using Model Context Protocol. The agent uses MCP-loaded tools instead of directly owning the database functions.

Learning outcomes

- Explain what MCP is and why it matters.
- Differentiate local tools from MCP-exposed tools.
- Build a local MCP server.
- Load MCP tools into LangChain.
- Run an MCP-powered customer-care agent.
- Discuss governance and safe tool boundaries.

Recommended delivery flow

- 1. Explain why notebook tools are good for learning but not enough for enterprise systems.
- 2. Introduce MCP tools, resources, and prompts.
- 3. Design a small safe customer-care tool set.
- 4. Create server file and DB helpers.
- 5. Load tools through client config.
- 6. Test one tool directly.
- 7. Run the MCP-powered agent.
- 8. Discuss safety and capstone mapping.

Transition note

Class 6 completes the technical architecture stack. Class 7 begins capstone planning using the patterns learned so far.

Class 6 MCP tool set

MCP tool	What it returns
get_customer_profile	Profile, city, segment, language, account status
get_customer_plan	Current subscriptions, plan, fee, data, minutes
get_customer_churn_risk	Churn score, risk level, reason, recommended action
get_customer_complaints	Recent complaints, severity, status, compensation
get_customer_billing_summary	Invoices, payment status, overdue days
get_customer_usage_summary	Data sessions, data used, usage cost
get_customer_value_segment	ARPU, six-month revenue, value segment

Code and prompt references from this class

Class 6 - MCP main demo prompt

Use the Zain Jordan MCP tools to analyze customer 42. Check profile, plan, churn risk, complaints, billing, usage, and value segment. Then recommend the next best action and draft a professional customer-care message.

Class 6 - MCP server build flow

1. Create server file: zain_customer_mcp_server.py
2. Import FastMCP
3. Define MCP server instance
4. Add DB helpers: connect, query, format output
5. Decorate tools with @mcp.tool()
6. Add optional resource or prompt
7. Run server using stdio transport

Slide-by-slide delivery notes - Class 6

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: MCP for Enterprise

AI Tool Integration | Exposing Zain Jordan Customer 360 tools through Model Context Protocol | MCP Server | LangChain MCP Client | Customer Care Agent | MCP | Standard protocol for tool and context access | Database Tools | Profile, churn, billing, usage, value

Slide 2: Where Class 6 Fits

JOURNEY | MCP comes after participants understand data access, tools, SQL, RAG, and multi-agent orchestration. | Database | Tools | SQL Agent | RAG | Multi-Agent | MCP | Capstone

Slide 3: Class 6 Goal

GOAL | Move from local notebook tools to enterprise-style tool access using MCP. | What participants will understand | - What MCP is | - Why MCP is useful | - How MCP differs from local tools | - How a client loads MCP tools | What participants will build | - Local MCP server

Slide 4: WHY MCP

Why MCP? | Notebook tools are good for demos; reusable tool servers are better for enterprise systems. | Without MCP | - Each app has custom integrations | - Tools are tightly coupled to one notebook | - Harder to reuse across teams | - Difficult to standardize access | - Harder to manage permissions | With MCP

Slide 5: CONCEPT

Conceptual Ladder | The class builds on what participants already know: functions, tools, and agents. | Python Function | LangChain Tool | Agent Uses Tool | MCP Server Exposes Tool | MCP Client Loads Tool | Agent Uses MCP Tool | Beginner translation

Slide 6: Before MCP vs With MCP

ARCHITECTURE | The same customer tools can be exposed in a more modular architecture. | Before MCP | Notebook / Python app | - Database functions inside notebook | - LangChain tools defined locally | - Agent tied to one codebase | - Good for fast learning | - Less reusable across applications

Slide 7: MCP Primitives

MCP BASICS | Class 6 focuses mainly on tools, with a brief introduction to resources and prompts. | Tools | Actions the model can call. | Zain examples: | - `get_customer_profile` | - `get_customer_churn_risk` | - `get_billing_summary` | Resources

Slide-by-slide delivery notes - Class 6 (continued)

Slide 8: Zain Jordan MCP Architecture

ARCHITECTURE | A local server exposes customer database capabilities; LangChain loads them as agent tools. | SQLite DB | Customer 360 tables | MCP Server | Exposes approved tools | MCP Client | Loads remote/local tools | Agent

Slide 9: Zain MCP Server Design

SERVER | Start with a small, safe set of high-value customer-care tools. | Version 1 tools | - get_customer_profile | - get_customer_churn_risk | - get_customer_billing_summary | Expanded tools | - get_customer_plan | - get_customer_complaints

Slide 10: MCP Tool Set: Customer 360

TOOLS | The server exposes customer-specific tools that the agent can safely call. | get_customer_profile | Profile, city, segment, language, account status | get_customer_plan | Current subscriptions, plan, fee, data, minutes | get_customer_churn_risk | Churn score, risk level, reason, recommended action | get_customer_complaints

Slide 11: Server Build Flow

BUILD | The notebook creates a local MCP server file and validates it before connecting. | 1. Create server file | zain_customer_mcp_server.py | 2. Import FastMCP | Define MCP server instance | 3. Add DB helpers | connect, query, format output | 4. Decorate tools

Slide 12: LangChain MCP Client Flow

CLIENT | The client launches the local server and loads MCP tools into LangChain. | Server config | command: python | args: server.py | MultiServer | MCPClient | connects to stdio server | get_tools()

Slide 13: AGENT

MCP-Powered Agent | The agent does not directly know the database; it only sees MCP-loaded tools. | Agent role | Professional Zain Jordan customer-care assistant. | Uses MCP tools whenever customer data is needed. | Agent rules | - Do not guess customer facts | - Use tools for data | - Say if data is missing

Slide 14: Main Demo Flow

DEMO | The demo proves that the LangChain agent can use MCP tools to produce a customer-care recommendation. | Demo 1 | Who is customer 42? | Demo 2 | Check churn risk using MCP tools. | Demo 3 | Full customer 360 analysis. | Demo 4

Slide-by-slide delivery notes - Class 6 (continued)

Slide 15: Expected Output

OUTPUT | The MCP-powered agent produces a structured customer-care report. | Customer Summary | Plan Summary | Churn Risk | Value Segment | Billing Summary | Complaints | Usage Summary

Slide 16: Testing and Debugging

DEBUG | Good MCP demos depend on a simple test sequence and clear failure points. | Test sequence | 1. Confirm database file exists | 2. Run normal SQL query | 3. Create MCP server file | 4. Syntax-check server | 5. Load tools | 6. Directly call one tool

Slide 17: Safety and Governance

GOVERNANCE | MCP should expose approved, controlled tools - not unrestricted access. | Training setup | - synthetic database | - local SQLite file | - selected read-only tools | - limited output rows | - classroom-only server | Production controls

Slide 18: Participant Exercises

PRACTICE | Learners connect MCP concepts to practical tool design and capstone architecture. | Exercise 1: Tool mapping | Map user requests to MCP tools. | Example: | "Is customer 42 likely to leave?" → `get_customer_churn_risk` | Exercise 2: Customer tests | Analyze customer 10, 25, 42, and 100 using MCP tools. Compare urgency and risk. | Exercise 3: Pattern choice

Slide 19: Capstone MCP Mapping

CAPSTONE | Teams decide which capabilities should be exposed through MCP. | Tool design questions | - What is the tool name? | - What input does it need? | - What output should it return? | - Who is the business user? | - What safety controls are needed? | Example tools

Slide 20: RISKS

Common MCP Mistakes | A strong MCP demo avoids these design and implementation errors. | Design mistakes | - Exposing too many tools | - Vague tool names | - Tool descriptions missing | - Tools with unclear outputs | - No ownership model | Technical mistakes

Slide 21: NEXT

What Comes Next: Capstone | After MCP, participants are ready to design and build their own AI solution. | Project brief | Team formation | Architecture map | Build sprint | Demo prep | Final presentation | Capstone expectation

Slide-by-slide delivery notes - Class 6 (continued)

Slide 22: SUMMARY

Class 6 Summary | MCP turns approved business capabilities into reusable tools for AI applications. | Built today | - Local MCP server | - Customer 360 MCP tools | - MCP client connection | - Agent using MCP tools | - Full customer analysis demo | Business value

10. Class 7 - Capstone Project Planning and Team Formation

This class changes the mode from instruction to project design. Students choose realistic telecom problems, define roles, select architecture, and prepare a build-ready blueprint.

Learning outcomes

- Choose a realistic capstone project.
- Define business problem, target user, and demo prompt.
- Map database tables and architecture pattern.
- Assign team roles.
- Prepare the minimum viable demo scope.

Recommended delivery flow

- 1. Review the learning journey.
- 2. Explain what makes a good capstone.
- 3. Present recommended capstone options.
- 4. Use architecture selection guide.
- 5. Assign team roles.
- 6. Fill the capstone blueprint.
- 7. Prepare Class 8 build plan.

Transition note

Class 7 creates the blueprint. Class 8 turns it into a working prototype.

Class 7 architecture selection guide

Project need	Best pattern
Retrieve one customer data	LangChain Tools or MCP Tools
Ask broad analytical questions	SQL Agent
Recommend plans, campaigns, or add-ons	RAG
Combine multiple specialist tasks	Multi-Agent
Expose tools in enterprise style	MCP
Draft customer messages	Care Message Agent

Slide-by-slide delivery notes - Class 7

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Capstone Project Planning and Team Formation

Designing AI Solutions with the Zain Jordan Customer 360 Database | Focus: Planning - Team Formation - Architecture - Build Readiness | Class Journey | 1. Database | 2. Tools | 3. SQL Agent | 4. RAG | 5. Multi-Agent | 6. MCP

Slide 2: Where We Are in the Journey

Class 1: Python + SQLite database exploration | Class 2: LangChain tools + single customer-care agent | Class 3: Natural Language SQL Agent | Class 4: RAG Assistant from telecom database content | Class 5: Multi-Agent Customer Care Copilot | Class 6: MCP for enterprise AI tool integration | Class 7: Capstone Project Planning and Team Formation | Today's Goal | Choose a realistic capstone project

Slide 3: What Makes a Good Capstone?

Keep the project practical, focused, and easy to explain.

Slide 4: Recommended Capstone Project Options

Teams can choose one of these or propose a similar telecom use case. | 1. Customer Care Copilot | 2. Churn Rescue Assistant | 3. Billing Support Assistant | 4. Plan and Add-on Recommendation Assistant | 5. Campaign Targeting Assistant | 6. Network Impact Service Recovery Assistant | 7. Executive KPI Insight Assistant | Selection Criteria

Slide 5: Architecture Selection Guide

Choose the architecture based on the type of question and workflow.

Slide 6: Team Formation and Roles

Clear role allocation helps the build sprint move faster. | Recommended Team Size | 3-5 participants per team | Small teams may combine roles | Each team should choose one team name | Each team should prepare one final demo owner | Suggested Roles | Business Lead - defines use case and target user | Data Lead - identifies tables and queries

Slide 7: Capstone Project Blueprint

1. Project Title | 2. Business Problem | 3. Target User | 4. Main User Prompt | 5. Database Tables Used | 6. AI Architecture | 7. Tools / Agents Needed | 8. Expected Output | 9. Demo Flow

Slide-by-slide delivery notes - Class 7 (continued)

Slide 8: Example: Churn Rescue Assistant Blueprint

A strong project is narrow enough to build and broad enough to show value.

Slide 9: Minimum Viable Demo

Do not try to build everything. | Build the smallest useful demo first. | A strong minimum demo should include: | One clear user prompt | Database-backed answer | At least one AI capability | Business recommendation | Simple 3-minute demo flow | Optional Enhancements

Slide 10: Team Activity Flow

This is the working sequence for the classroom activity.

Slide 11: Evaluation Criteria

Use these criteria to guide both planning and final presentation.

Slide 12: Final Presentation Structure

Recommended team presentation length: 5 minutes.

Slide 13: Class 7 Deliverables

1. Team name | 2. Team members and roles | 3. Project title | 4. Business problem and target user | 5. Selected database tables | 6. Chosen AI architecture | 7. Main demo prompt | 8. Expected output | 9. Build plan for Class 8

11. Class 8 - Capstone Build Sprint

This class turns capstone ideas into working prototypes. Students focus on one working prompt, one database-backed output, one useful recommendation, and one demo story.

Learning outcomes

- Choose a build track.
- Build the smallest useful demo first.
- Validate one working prompt.
- Generate a database-backed answer.
- Add a business recommendation.
- Prepare final presentation structure.

Recommended delivery flow

- 1. Confirm scope.
- 2. Load data.
- 3. Choose build track.
- 4. Build and test prompt.
- 5. Validate output.
- 6. Prepare final demo report.
- 7. Use trainer checkpoints for scope, data, architecture, demo, and presentation.

Transition note

Class 8 produces working prototypes. Class 9 evaluates and reviews them professionally.

Class 8 build tracks

Track	Best for
Track A - SQL Agent	Rankings, counts, KPI summaries, business insights
Track B - Tools Agent	Customer lookup, billing, churn, Customer 360
Track C - RAG	Plan offers, campaigns, add-ons, complaint themes
Track D - Multi-Agent	Customer data + SQL analysis + RAG + message agent
Track E - MCP	Enterprise tools and reusable access, optional advanced path

Code and prompt references from this class

Class 8 - Final demo output template

```
Project title
Business problem
Target user
Selected track
Database tables used
Main demo prompt
Final AI output
Data-backed evidence
Business interpretation
Recommended action
Limitations or missing data
Next-step improvement
```

Slide-by-slide delivery notes - Class 8

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Capstone Build Sprint

From project blueprint to working AI prototype | Focus: Build Tracks - Working Prompt - Database-Backed Output - Demo Readiness | Build Sprint | SQL Agent | Tools Agent | RAG | Multi-Agent | MCP | Final Demo

Slide 2: Where We Are in the Journey

Class 8 moves teams from planning to execution. | Learning Journey | Class 1: Database exploration | Class 2: LangChain tools + single agent | Class 3: Natural Language SQL Agent | Class 4: RAG Assistant | Class 5: Multi-Agent Customer Care Copilot | Class 6: MCP for enterprise tool integration | Class 7: Capstone planning and team formation

Slide 3: Build Sprint Rules

The goal is a strong minimum demo first, not a perfect production system.

Slide 4: Choose Your Build Track

Teams can choose one or combine tracks, but should start with one working path. | Track A | SQL Agent | Rankings | Counts | KPI summaries | Business insights | Track B | Tools Agent

Slide 5: Recommended Sprint Flow

This sequence keeps teams focused and avoids overbuilding. | Confirm Scope | Load Data | Choose Track | Build Prompt | Test Output | Prepare Demo | Sprint Principle | Build the smallest useful demo first.

Slide 6: Track A: SQL Agent Projects

Use SQL Agent when the project needs calculations, rankings, grouping, or summaries.

Slide 7: Track B: Tools-Based Customer Agent

Use Tools Agent when the project needs controlled customer lookup.

Slide-by-slide delivery notes - Class 8 (continued)

Slide 8: Track C: RAG Recommendation Projects

Use RAG when the answer needs semantic search, explanation, or recommendation.

Slide 9: Track D: Multi-Agent Projects

Use Multi-Agent when the capstone has multiple specialist tasks.

Slide 10: Track E: MCP-Enhanced Projects

MCP is optional for Class 8 and best for confident teams.

Slide 11: Final Demo Output Template

Every team should produce a clean final response. | Demo Report | Required sections | Project title | Business problem | Target user | Selected track | Database tables used | Main demo prompt

Slide 12: Presentation Checklist

Use this before the final class to avoid last-minute confusion.

Slide 13: Trainer Sprint Checkpoints

Use these checkpoints during the build sprint.

Slide 14: Class 8 Deliverables

Each team should finish with a build-ready demo package. | Team Deliverables | By the end of Class 8 | Working notebook or prototype | One successful demo prompt | Database-backed answer | AI-generated recommendation or summary | Final output example | 5-minute presentation draft

12. Class 9 - Final Capstone Presentations and Evaluation

The final class is a solution review, not a new technical module. Students present their project, demonstrate the prototype, receive feedback, and connect the work to enterprise AI next steps.

Learning outcomes

- Present the capstone project.
- Demonstrate the working prototype.
- Explain data and architecture choices.
- Receive structured feedback.
- Identify next improvements.

Recommended delivery flow

- 1. Open with expectations.
- 2. Explain demo rules and scoring.
- 3. Give teams final preparation time.
- 4. Run team presentations.
- 5. Capture feedback and scoring.
- 6. Recap the learning journey.
- 7. Discuss next-step improvements and close.

Transition note

Class 9 closes the applied AI journey from database understanding to AI solution review.

Class 9 evaluation rubric

Criteria	Weight	Guiding question
Business Relevance	25	Does the project solve a meaningful telecom problem?
Correct Use of Database	25	Does it use the Zain Jordan Customer 360 database correctly?
AI Architecture Clarity	25	Did the team choose and explain the right AI pattern?
Demo Quality and Explanation	25	Is the demo clear, structured, convincing, and reliable?

Slide-by-slide delivery notes - Class 9

Use this section while teaching. It keeps the delivery aligned to the actual class deck and helps connect each slide to the larger workshop story.

Slide 1: Final Capstone Presentations and Evaluation

Showcasing AI applications built with the Zain Jordan Customer 360 database | Focus: Final Demos - Evaluation - Feedback - Next Steps | Final Showcase | Team Demos | Evaluation | Feedback | Awards | Next Steps

Slide 2: Full Learning Journey

Class 9 closes the journey from database understanding to AI solution review. | Database | Tools | SQL Agent | RAG | Multi-Agent | MCP | 7-8 | Capstone

Slide 3: Class 9 Objectives

The final class is a solution review, not a new technical module. | What Teams Will Do | Present the capstone project | - Demonstrate the working prototype | - Explain data and architecture choices | - Receive structured feedback | - Identify next improvements | What Judges Will Review | Problem clarity

Slide 4: Recommended Session Flow

Adjust timing based on number of teams.

Slide 5: Team Presentation Structure

Each team should present in a clear 5-minute story.

Slide 6: Evaluation Rubric

Recommended scoring: 100 points total.

Slide 7: Scoring Guide

Use this to keep judging consistent across teams.

Slide-by-slide delivery notes - Class 9 (continued)

Slide 8: Judge Feedback Framework

Feedback should be specific, constructive, and actionable. | What Worked Well | Business problem clarity | - Good use of database | - Strong architecture choice | - Useful recommendation | - Confident demo flow | What to Improve | Prompt clarity

Slide 9: Live Demo Readiness Checklist

Use this before each team starts presenting. | Technical Checks | Notebook or prototype opens | - API key or environment is ready | - Database file is available | - Main demo prompt runs | - Final output is saved or visible | Story Checks | Problem is clear

Slide 10: Award Categories

Optional categories can recognize different types of excellence.

Slide 11: Final Evaluation Outputs

The evaluation notebook supports a professional closing process. | Notebook Outputs | Team registration table | - Evaluation rubric | - Score entry table | - Automatic rankings | - Award suggestions | - Judge feedback templates | Exported Files

Slide 12: Next-Step Recommendations

Teams can turn prototypes into stronger enterprise AI solutions.

Slide 13: Closing Message

Class 9 completes the full applied AI learning journey. | What Participants Practiced | Business problem definition | - Database understanding | - AI architecture selection | - Prototype building | - Live demonstration | - Feedback-driven improvement | Final Takeaway

13. Prompt and Code Repository

Collected from the workshop decks and Codex notes for quick use during the three-day session.

This section collects the key code snippets and prompt patterns used across the workshop. It is designed as a quick-reference appendix for trainers and students.

1. Class 1 - Environment setup

```
import sqlite3
import pandas as pd
from pathlib import Path
import os

DB_PATH = "zain_customer_360_ai_demo.db"
conn = sqlite3.connect(DB_PATH)
print("Connected successfully")
```

2. Class 1 - Inspect database tables

```
query = """
SELECT name
FROM sqlite_master
WHERE type = 'table'
ORDER BY name;
"""

tables_df = pd.read_sql_query(query, conn)
tables_df
```

3. Class 1 - First business join

```
SELECT
  c.customer_id,
  c.full_name,
  c.city,
  s.msisdn,
  s.service_type,
  p.plan_name,
  p.monthly_fee_jod
FROM customers c
JOIN subscriptions s ON c.customer_id = s.customer_id
JOIN plans p ON s.plan_id = p.plan_id
LIMIT 10;
```

4. Class 1 - Reusable customer profile function

```
def get_customer_profile(customer_id):
    query = """
    SELECT c.customer_id, c.full_name,
           c.city, c.customer_segment,
           c.preferred_language, c.status
    FROM customers c
    WHERE c.customer_id = ?
    """
    return pd.read_sql_query(query, conn, params=(customer_id,))
```

5. Class 1 - First Customer 360 function

```
def get_customer_360(customer_id):  
    return {  
        "profile": get_customer_profile(customer_id),  
        "plans": get_customer_plan(customer_id),  
        "churn": get_customer_churn_risk(customer_id),  
        "complaints": get_customer_complaints(customer_id),  
        "support": get_customer_support_interactions(customer_id)  
    }
```

Prompt and Code Repository (continued)

6. Class 2 - Convert function into LangChain tool

```
@tool
def get_customer_churn_risk(customer_id: int) -> str:
    """Get churn score, risk level, reason, and recommended action."""
    # Call database function and return readable output
    return fetch_customer_churn_risk(customer_id)
```

7. Class 2 - Main Customer Care Agent prompt

Analyze customer 42. Check profile, plan, churn risk, recent complaints, support history, billing, and usage. Recommend the next best action and draft a short message.

8. Class 2 - First two-tool demo prompt

Who is customer 42 and what is their churn risk?

9. Class 3 - Manual SQL warm-up

```
SELECT c.city, COUNT(*) AS high_risk_customers
FROM customers c
JOIN customer_churn_scores ch
    ON c.customer_id = ch.customer_id
WHERE ch.risk_level = 'High'
GROUP BY c.city
ORDER BY high_risk_customers DESC
LIMIT 10;
```

10. Class 3 - SQL Agent rules prompt

You are a telecom BI SQL agent.

Rules:

- Use SELECT queries only.
- Never modify the database.
- Inspect schemas before querying.
- Limit results unless requested.
- Do not guess missing facts.
- Explain results in business language.

Final answer:

1. Direct answer
2. Key numbers
3. Business interpretation
4. Recommended next action

Prompt and Code Repository (continued)

11. Class 4 - Build documents from database rows

```
for row in plans:
    text = f"""
    Plan: {row.plan_name}
    Category: {row.plan_category}
    Monthly Fee: {row.monthly_fee_jod} JOD
    Data: {row.data_allowance_gb} GB
    Minutes: {row.local_minutes}
    """
    documents.append(Document(
        page_content=text,
        metadata={"table": "plans", "id": row.plan_id}
    ))
```

12. Class 4 - Create embeddings and vector store

```
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = InMemoryVectorStore(embeddings)
vector_store.add_documents(documents)
retriever = vector_store.as_retriever(search_kwargs={"k": 4})
```

13. Class 4 - Retrieval before generation

```
query = "best offer for a heavy data user with roaming needs"
results = retriever.invoke(query)

for doc in results:
    print(doc.metadata)
    print(doc.page_content[:500])
```

14. Class 4 - RAG answer format

```
Final answer format:
1. Recommended plan / add-on / campaign
2. Why it fits the customer need
3. Evidence from retrieved documents
4. Confidence / limitations
5. Customer-care message
```

Prompt and Code Repository (continued)

15. Class 5 - Multi-agent demo prompt

Analyze customer 42. Check profile, plan, churn risk, value, billing, complaints, support history, usage, relevant offers, then draft a professional care message.

16. Class 6 - MCP main demo prompt

Use the Zain Jordan MCP tools to analyze customer 42. Check profile, plan, churn risk, complaints, billing, usage, and value segment. Then recommend the next best action and draft a professional customer-care message.

17. Class 6 - MCP server build flow

1. Create server file: zain_customer_mcp_server.py
2. Import FastMCP
3. Define MCP server instance
4. Add DB helpers: connect, query, format output
5. Decorate tools with @mcp.tool()
6. Add optional resource or prompt
7. Run server using stdio transport

18. Class 8 - Final demo output template

Project title
Business problem
Target user
Selected track
Database tables used
Main demo prompt
Final AI output
Data-backed evidence
Business interpretation
Recommended action
Limitations or missing data
Next-step improvement

14. Codex Dashboard Demo Playbook

Use this as the live software-building demonstration with the same Customer 360 database.

The recommended Codex demo project is the Zain Customer Churn Risk Dashboard. It is intentionally simple enough for beginners but still business-relevant for a telecom client. The purpose is to show how Codex can create, debug, improve, and package a small database-backed application.

Area	Detail
Project	Zain Customer Churn Risk Dashboard
User	Retention manager or customer-care manager
Business problem	The retention team wants to identify high churn-risk customers and understand why they need action.
Architecture	SQLite database -> Python SQL queries -> Pandas DataFrame -> Streamlit dashboard -> retention insight
File structure	app.py, db_utils.py, requirements.txt, README.md, data/zain_customer_360_ai_demo.db

Main Codex prompt

```
I have a SQLite database called zain_customer_360_ai_demo.db inside a data folder.

Build a simple Streamlit app called "Zain Customer Churn Risk Dashboard".

Requirements:
1. Connect to the SQLite database.
2. Show four KPI cards:
  - total customers
  - high-risk churn customers
  - average churn score
  - average ARPU
3. Add filters for risk level and city.
4. Show a table of customers with:
  - customer_id
  - full_name
  - city
  - customer_segment
  - churn_score
  - risk_level
  - main_risk_reason
  - recommended_action
5. When a user selects a customer_id, show a Customer 360 summary:
  - profile
  - plan
  - churn risk
  - value segment
  - recent complaints
  - support history
6. Add a short retention recommendation section.
7. Keep the code beginner-friendly and split database functions into db_utils.py.
8. Create requirements.txt and README.md.
```

Live demo sequence

- Step 1 - Ask Codex to inspect the project: Tell me what files are in this project and what this app currently does.
- Step 2 - Ask Codex to build the dashboard using the main prompt.

- Step 3 - Run the app: `streamlit run app.py`
- Step 4 - Improve layout: Add clear section headers, better KPI cards, and a CSV download button.
- Step 5 - Debug safely: The app is giving this error. Please find the cause and fix it with the smallest safe change.

Codex task map

Codex task	What to ask Codex to do
Create project files	Create a Streamlit app for this SQLite database.
Write SQL queries	Add a query to show high-risk churn customers.
Build UI	Add dropdown filters for risk level and city.
Debug errors	Fix this SQLite column error.
Improve UX	Make the dashboard cleaner and add KPIs.
Add export	Add a CSV download button.
Refactor code	Move SQL queries into helper functions.

Minimum version for a short live demo

- Upload or connect the SQLite database.
- Filter high-risk customers.
- Display the customer table.
- Select `customer_id`.
- Show customer churn summary.
- Then ask Codex to improve the app step by step.

15. Capstone and Evaluation Reference

Use this section for team planning, build sprint, final presentations, and judging.

Blueprint item	What the team must define
Project Title	Name of the AI application.
Business Problem	The telecom issue the project solves.
Target User	The user who will use the tool.
Main User Prompt	The primary demo prompt.
Database Tables Used	Tables required to ground the answer.
AI Architecture	Tools, SQL Agent, RAG, Multi-Agent, MCP, or combination.
Expected Output	What the final answer/application should produce.
Demo Flow	How the team will show the solution in 3-5 minutes.
Success Criteria	How the team knows the demo is useful and grounded.

Final presentation structure

Presentation step	What to present
1. Project Title	Name of the AI application
2. Business Problem	What issue are you solving?
3. Target User	Who will use the tool?
4. Data Used	Which Zain Jordan tables are used?
5. AI Architecture	Tools, SQL Agent, RAG, Multi-Agent, MCP
6. Live Demo	Show the working notebook or prototype
7. Business Value	Why the solution matters
8. Future Improvements	What you would add next

Closing Summary

The complete workshop story in one page.

The Zain Jordan Customer 360 AI Workshop is a complete applied AI journey. It begins with structured telecom data, converts database functions into AI tools, expands into natural-language SQL analytics, adds RAG for semantic recommendations, orchestrates specialist agents, exposes reusable capabilities through MCP, and ends with capstone projects that are evaluated for business value and technical clarity.

Final architecture progression

- Database understanding
- Reusable Python functions
- LangChain tools
- Customer Care Agent
- SQL Agent
- RAG Assistant
- Multi-Agent Customer Care Copilot
- MCP enterprise tool integration
- Capstone solution design
- Final evaluation and feedback

The goal is not only to teach tools. The goal is to help participants understand how enterprise AI applications are designed, grounded, tested, explained, and improved.